Tutoriat LMC

Vlăduț Ghiță, Raluca Ionete, Voaideș Robert

April 1, 2026

Contents

1 Introducere în LP și Prolog **2**
- 1.1 Administrativ . . . 2
- 1.2 De ce Logică? . . . 2
- 1.3 Logica Propozițională - intro . . . 3
  - 1.3.1 Principiul inducției pe formule . . . 4
  - 1.3.2 Principiul recursiei pe formule . . . 5
- 1.4 Semantica LP . . . 6
  - 1.4.1 Evaluări . . . 6
  - 1.4.2 Modele. Tautologii . . . 7
- 1.5 Prolog Basics . . . 10

2 Continuare Prolog, FNC si FND **13**
- 2.1 Funcții Boolene . . . 13
- 2.2 Forma normală Conjunctivă și Forma Normală Disjunctivă . . . 14
  - 2.2.1 Concepte Fundamentale . . . 14
  - 2.2.2 FNC . . . 14
  - 2.2.3 FND . . . 14
  - 2.2.4 Aducerea formulelor în FNC și FND . . . 15
  - 2.2.5 Algoritmul de construcție pentru FNC și FND de la un tabel . . . 15
  - 2.2.6 Algoritmul de aducere a unei formule la FND/FNC . . . 16
- 2.3 Prolog Aritmetica si Exercitii . . . 18
- 2.4 Prolog findall/setof/bagof . . . 22

Chapter 1

Introducere în LP și Prolog

1.1 Administrativ

- Cursurile d-lui Leuștean sunt foarte bine scrise și chiar merită citite pentru a învăța la materia asta. Le găsiți pe toate pe GitHub. În plus, la examen **o să fie nevoie** să folosiți propoziții din curs (care sunt numerotate), și să le citați, așa că vă ajută să știți cam care este structura cursului. Nu o să reluăm teoria pe care o prezintă în curs **decât** dacă ne cereți asta.
- Seminariile sunt destul de complexe, acoperă ok materia, iar soluțiile sunt postate pe site, așa că noi o să încercăm să lucrăm la tutoriat exerciții extra.
- Puteți să ne scrieți oricând în privat/pe grup cu întrebări/sugestii/idei, iar noi o să încercăm să vă ajutăm și să le punem în practică. De asemenea, puteți să ne sugerați ce ați refera să lucrăm la tutoriat.
- O să publicăm pe GitHub tutoriatele săptămânal și o să anunțăm, prin README, ce o să facem la următorul tutoriat. De asemenea, plănuim să creăm liste de exerciții, la care să publicăm și rezolvările. Dacă o să doriți să ne uităm pe rezolvările voaste, puteți să ne ziceți.

1.2 De ce Logică?

Este foarte normal să vă întrebați de ce e nevoie să studiați logica ca studenți la info, având în vedere că e aproape matematică pură, care nu pare să aibă foarte mare aplicabilitate în viața reală.
Well, sunt mai multe motive.

Logica e baza informaticii

Aproape toți oamenii care au fondat informatica erau logicieni. Tot ce înseamnă informatică formală se poate traduce în logică, literally.

Asta se traduce în două lucruri:

- nu o să scăpați de logică pentru încă ceva timp, în anul II aveți două materii care se construiesc pe logică și la care o să vă ajute foarte mult să aveți bazele bine stabilite.
- felul foarte formal de a gândi care este promovat de logică vă va ajuta foarte mult în a deveni programatori mai buni; o să ajungeți să gândiți mult mai mult ce scrieți, atât în matematică cât și în programare, pentru a fi mult mai riguros și mai logic exprimat, ceea ce o să vă ajute mai ales atunci când aveți lucruri mai complicate de făcut.

Logica vă deschide drumul și către o ramură destul de nișată a informaticii, dar care este foarte bine plătită și foarte AI-proof, anume verificarea formală, care, pe scurt, presupune demonstrarea matematică a corectitudinii unui cod, lucru care este esențial în sisteme high-risk (procesoare, sisteme medicale, rachete etc.) Facultatea noastră este foarte bogată în logicieni, se țin foarte des seminarii de logică la ILDS sau LOS, așa că dacă vreți să faceți cercetare în timpul facultății sau să vă specializați pe partea de verificare formală sau alte tipuri de logică, chiar aveți cu cine.

1.3 Logica Propozițională - intro

Formulele stau la baza Logicii Propoziționale și sunt principalul lucru cu care o să lucrați capitolul ăsta. Cursul menționează că ele sunt definite inductiv:

Definition 1 (Formule). Formulele sunt definite astfel:

(1) Orice variabilă propozițională este formulă. (Adică $\forall v \in V$, v este formulă)

(2) Dacă φ este formulă, atunci $\neg\varphi$ este formulă

(3) Dacă φ, ψ sunt formule, atunci $\psi \to \varphi$ este formulă

Dar de ce este inductiv? Până acum, ați folosit doar inducția pe numere naturale, care știți cum funcționa. Aveam un caz de bază $P(0)$ și, presupunând că predicatul era adevărat pentru un n natural, construiam următorul caz, $n+1$, și demonstram corectitudinea.

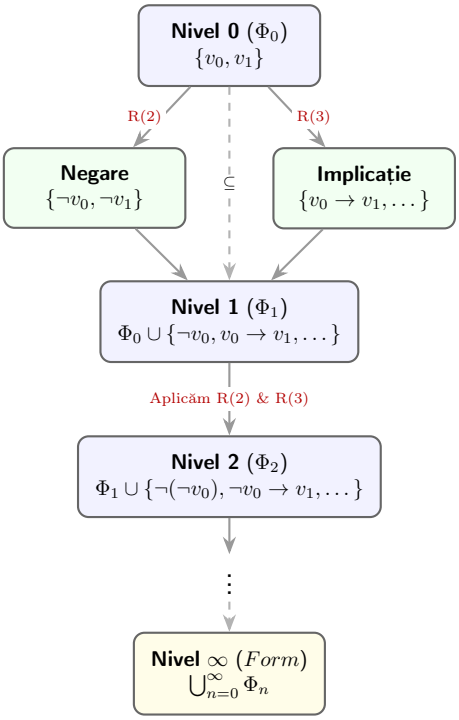
Practic, inducția reprezintă metoda de a pleca de la ceva particular și de a generaliza incremental. Pentru numere naturale, pasul incremental este simplu, anume adunarea.

Noi putem extinde metoda asta, astfel, și la formule. Avem cazul particular, pe care îl definim a fi formulă, adică variabilele v. De aici, incremental, construim toate celelalte formule. În loc de un singur pas incremental, avem doi:

$\neg$ putem nega formula noastră curentă pentru a obține o nouă formulă

$\to$ atunci când avem două formule, o nouă formulă este definită prin implicație

Această metodă se numește **inducție structurală**.
În următoarea figură puteți vedea cum sunt construite formulele din doar două variabile.



1.3.1 Principiul inducției pe formule

Același principiu poate fi a aplicat pentru a demonstra că orice formulă respectă o proprietate $\mathbf{P}$ (e.g., orice formulă are mai $\leq$ conectori logici decât variabile, orice formulă este bine parantezată, etc.).

Proposition 1. Fie $\mathbf{P}$ o proprietate. Presupunem că:

(0) Orice variabilă v are proprietatea $\mathbf{P}$.

(1) Pentru orice formulă φ, dacă φ are proprietatea $\mathbf{P}$, atunci $\neg\varphi$ are proprietatea $\mathbf{P}$.

(2) Pentru orice formule φ, ψ, dacă acestea au proprietatea $\mathbf{P}$, atunci $\varphi \to \psi$ are proprietatea $\mathbf{P}$.

Atunci orice formulă φ are proprietatea $\mathbf{P}$.

Example. Vrem să demonstrăm că orice formulă φ este parantezată corect. Formal: orice formulă φ are proprietatea $\mathbf{P}(\varphi) : st(\varphi) = dr(\varphi)$, unde $st(\varphi)$ este numărul de paranteze "(", iar $dr(\varphi)$ este numărul de paranteze ")".

Proof. Demonstrăm proprietatea $P(\varphi)$ prin inducție structurală după construcția formulei φ.

Cazul de bază: $\varphi = v$, unde $v \in V$ este o variabilă propozițională. Conform definiției sintaxei, variabilele nu conțin paranteze.

$$st(v) = 0 \text{ și } dr(v) = 0 \Rightarrow st(v) = dr(v)$$

Deci $P(v)$ este adevărată.

Pasul inductiv: Presupunem că proprietatea este adevărată pentru formulele ψ și χ (Ipoteza Inductivă). Deci $st(\psi) = dr(\psi)$ și $st(\chi) = dr(\chi)$.

Demonstrăm pentru regulile de construcție:

- **Cazul** $\varphi = (\neg\psi)$: Calculăm numărul de paranteze folosind definiția recursivă:

$$st(\varphi) = st(\psi) + 1$$
$$dr(\varphi) = dr(\psi) + 1$$

  Din Ipoteza Inductivă știm că $st(\psi) = dr(\psi)$. Adunând 1 în ambele părți, obținem $st(\varphi) = dr(\varphi)$.

- **Cazul** $\varphi = (\psi \to \chi)$: Calculăm numărul de paranteze:

$$st(\varphi) = 1 + st(\psi) + st(\chi)$$
$$dr(\varphi) = dr(\psi) + dr(\chi) + 1$$

  Înlocuind din Ipoteza Inductivă ($st \to dr$), avem:

$$st(\varphi) = 1 + dr(\psi) + dr(\chi)$$

  Datorită comutativității adunării, $1 + dr(\psi) + dr(\chi) = dr(\psi) + dr(\chi) + 1$, deci $st(\varphi) = dr(\varphi)$.

Conform principiului inducției structurale, proprietatea $P(\varphi)$ este adevărată pentru orice formulă φ. □

1.3.2 Principiul recursiei pe formule

Dacă inducția ne ajută să demonstrăm proprietăți, principiul recursiei pe formule este folosit pentru a da **definiții recursive** riguroase diverselor funcții asociate formulelor.

Proposition 2 (Principiul recursiei pe formule)**.** Fie A o mulțime și funcțiile:

$$G_0 : V \to A, \quad G_\neg : A \to A, \quad G_\to : A \times A \to A$$

Atunci există o **unică** funcție $F : Form \to A$ care satisface următoarele

proprietăți:

(R0) $F(v) = G_0(v)$ pentru orice variabilă $v \in V$.

(R1) $F(\neg\varphi) = G_\neg(F(\varphi))$ pentru orice formulă φ.

(R2) $F(\varphi \to \psi) = G_\to(F(\varphi), F(\psi))$ pentru orice formule φ, ψ.

Example. Vrem să definim o funcție $var : Form \to \mathcal{P}(V)$ care extrage mulțimea tuturor variabilelor propoziționale ce apar într-o formulă φ (unde $\mathcal{P}(V)$ este mulțimea părților lui V). O definiție recursivă a lui var este următoarea:

$$\begin{aligned} var(v) &= \{v\} && \text{pentru orice variabilă } v \\ var(\neg\varphi) &= var(\varphi) && \text{pentru orice formulă } \varphi \\ var(\varphi \to \psi) &= var(\varphi) \cup var(\psi) && \text{pentru orice formule } \varphi, \psi \end{aligned}$$

În acest caz, maparea pe principiul general se face astfel:

- Mulțimea A este $\mathcal{P}(V)$.
- $G_0 : V \to \mathcal{P}(V)$, definită prin $G_0(v) = \{v\}$.
- $G_\neg : \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\neg(X) = X$ (negația nu modifică variabilele existente).
- $G_\to : \mathcal{P}(V) \times \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\to(X, Y) = X \cup Y$ (reunim variabilele din ambele subformule).

1.4 Semantica LP

În acest capitol se studiază semantica, adică, informal, **ce înseamnă o formulă/cum îi dăm sens**. Acest lucru se face prin intermediul **evaluărilor** (sau interpretărilor, dar mai rar), care sunt funcții ce **dau valoare variabilelor propoziționale**.

1.4.1 Evaluări

O evaluarea este orice funcție:

$$e : V \to \{0, 1\}$$

Cu ajutorul recursiei pe formule este foarte ușor să extindem de la evaluări (care sunt pe variabile), la o funcție care interpretează formule:

Theorem 1. Pentru orice evaluare $e : V \to \{0, 1\}$ există o unică funcție (vezi recursia pe formule):

$$e^+ : Form \to \{0,1\}$$

care verifică următoarele proprietăți

(1) $e^+(v) = e(v), \quad \forall v \in V$

(2) $e^+(\neg\varphi) = \neg e^+(\varphi), \quad \forall \varphi \in Form$

(3) $e^+(\varphi \to \psi) = e^+(\varphi) \to e^+(\psi), \quad \forall \varphi, \psi \in Form$

Două cum puteți vedea, conectorii logici $(\neg, \to)$ sunt îngroșați atunci când sunt scoși din e^+. Acest lucru este pentru a arăta că aceștia nu mai sunt simboluri, în cadrul formulelor, ci **operații** pe $\{0,1\}$, cu tabele de adevăr știute.

Proposition 3. Dacă două evaluări sunt identice pentru toate variabilele dintr-o formulă φ, atunci ele vor da aceeași evaluare a formulei.

$$e_1(v) = e_2(v)\ \forall v \in Var(\varphi) \Rightarrow e_1^+(\varphi) = e_2^+(\varphi)$$

Propoziția 3 este utilă prin faptul că, atunci când evaluăm formule, putem să ne restrângem doar la variabilele din formulele pe care le evaluăm. Acest lucru se traduce în **metoda tabelului**, în care fiecare linie a tabelului este o evaluare diferită a variabilelor din $Var(\varphi)$.

1.4.2 Modele. Tautologii

Definition 2. Fie o evaluare $e : V \to \{0,1\}$.

Model: e este **model** pentru o formulă φ dacă $e^+(\varphi) = 1$. Notăm $e \models \varphi$.

Satisfiabilitate: O formulă φ se numește **satisfiabilă** dacă există cel puțin o evaluare e care este model pentru φ (adică $\exists e$ a.î. $e \models \varphi$).

Nesatisfiabilitate: O formulă φ este **nesatisfiabilă** (sau **contradictorie**) dacă nu admite niciun model (adică pentru orice evaluare e, $e^+(\varphi) = 0$). Notăm uneori prin $\varphi \models \bot$.

Tautologie: O formulă φ este **tautologie** dacă orice evaluare este model pentru ea (adică $\forall e, e \models \varphi$). Notăm acest lucru prin $\models \varphi$.

Notation. Mulțimea tuturor modelelor unei formule φ se notează cu $Mod(\varphi)$.

$$Mod(\varphi) = \{e : V \to \{0,1\} \mid e \models \varphi\}$$

Toți termenii definiți în 2 sunt foarte importanți și trebuie reținuți, pentru că o să tot lucrați cu ei.

Example. Formula $p \vee \neg p$ este o **tautologie**, deoarece indiferent de valoarea lui p, formula va fi adevărată. În schimb, $p \wedge \neg p$ este o **contradicție**, deoarece nicio evaluare nu o poate face adevărată.

Proposition 4 (Legile lui De Morgan). Pentru orice formule $\varphi, \psi \in Form$, au loc următoarele echivalențe logice:

$$\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi \tag{10}$$
$$\neg(\varphi \wedge \psi) \sim \neg\varphi \vee \neg\psi \tag{11}$$

Demonstrație:

Vom demonstra prima lege: $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. Demonstrația celei de-a doua legi se realizează în mod absolut analog.

(i) Metoda tabelului de adevăr

Fie $e : V \to \{0, 1\}$ o evaluare oarecare. Construim tabelul de adevăr pentru a evalua extensia e^+ pentru ambele formule. Coloanele de la stânga liniei verticale reprezintă valorile posibile pentru $e^+(\varphi)$ și $e^+(\psi)$, iar în dreapta evaluăm pas cu pas subformulele.

φ	ψ	$\varphi \vee \psi$	$\neg(\varphi \vee \psi)$	$\neg\varphi$	$\neg\psi$	$\neg\varphi \wedge \neg\psi$
1	1	1	0	0	0	0
1	0	1	0	0	1	0
0	1	1	0	1	0	0
0	0	0	1	1	1	1

Analizând tabelul, observăm că pentru toate cele 4 cazuri posibile ale unei evaluări e, valoarea sub coloana $\neg(\varphi \vee \psi)$ este identică cu valoarea sub coloana $\neg\varphi \wedge \neg\psi$. Astfel, $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e, ceea ce înseamnă că au exact aceleași modele. Prin urmare, $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$.

(ii) Metoda prin raționament semantic (formal)

Raționăm direct. Pentru o evaluare arbitrară $e : V \to \{0, 1\}$, observăm că:

$$e^+(\neg(\varphi \vee \psi)) = \neg(e^+(\varphi) \vee e^+(\psi)),$$
$$e^+(\neg\varphi \wedge \neg\psi) = \neg e^+(\varphi) \wedge \neg e^+(\psi).$$

Avem cazurile:

(a) $e^+(\varphi) = 0$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(0 \vee e^+(\psi)) = \neg e^+(\psi),$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 0 \wedge \neg e^+(\psi) = 1 \wedge \neg e^+(\psi) = \neg e^+(\psi).$$

(b) $e^+(\varphi) = 1$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(1 \vee e^+(\psi)) = \neg 1 = 0,$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 1 \wedge \neg e^+(\psi) = 0 \wedge \neg e^+(\psi) = 0.$$

Deoarece în ambele cazuri rezultatele evaluărilor coincid, obținem că $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e. Rezultă $Mod(\neg(\varphi \vee \psi)) = Mod(\neg\varphi \wedge \neg\psi)$, deci $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. □

Exercițiu. Să se găsească câte un model (o evaluare de adevăr) pentru fiecare dintre următoarele formule:

(i) $\varphi_1 = (v_1 \vee v_2) \to \neg v_3$

(ii) $\varphi_2 = (v_0 \wedge v_1) \wedge \neg(v_1 \to v_2)$

(iii) $\varphi_3 = v_5 \leftrightarrow (v_6 \wedge \neg v_5)$

Soluție. Reamintim că un model pentru o formulă φ este o funcție de evaluare $e : V \to \{0, 1\}$ astfel încât $e^*(\varphi) = 1$.

(i) Pentru $\varphi_1 = (v_1 \vee v_2) \to \neg v_3$: O implicație este adevărată dacă antecedentul este fals sau consecventul este adevărat. Alegem un model e în care $e(v_1) = 0$ și $e(v_2) = 0$. Atunci:

$$e^+((v_1 \vee v_2) \to \neg v_3) = (0 \vee 0) \to e^+(\neg v_3) = 0 \to e^+(\neg v_3) = 1$$

Deci, un model posibil este orice funcție e cu $e(v_1) = 0, e(v_2) = 0$ (valorile pentru v_3 și restul variabilelor pot fi arbitrare).

(ii) Pentru $\varphi_2 = (v_0 \wedge v_1) \wedge \neg(v_1 \to v_2)$: Pentru ca o conjuncție să fie adevărată, ambii termeni trebuie să fie adevărați.

1. $e^+(v_0 \wedge v_1) = 1 \Rightarrow e(v_0) = 1$ și $e(v_1) = 1$.
2. $e^+(\neg(v_1 \to v_2)) = 1 \Rightarrow e^+(v_1 \to v_2) = 0 \Rightarrow e(v_1) = 1$ și $e(v_2) = 0$.

Modelul este $e(v_0) = 1, e(v_1) = 1, e(v_2) = 0$.

(iii) Pentru $\varphi_3 = v_5 \leftrightarrow (v_6 \wedge \neg v_5)$: Echivalența este adevărată dacă ambele părți au aceeași valoare de adevăr. Dacă presupunem $e(v_5) = 1$, atunci trebuie ca $e^+(v_6 \wedge \neg v_5) = 1$, ceea ce implică $e^+(\neg v_5) = 1 \Rightarrow e(v_5) = 0$ (contradicție). Dacă presupunem $e(v_5) = 0$, atunci trebuie ca $e(v_6 \wedge \neg v_5) = 0$. Cum $e^+(\neg v_5) = 1$, conjuncția $e^+(v_6 \wedge 1)$ este 0 dacă și numai dacă $e(v_6) = 0$. Deci, un model este $e(v_5) = 0, e(v_6) = 0$.

□

1.5 Prolog Basics

Prologul are la baza **rezolutia**, despre care o sa invatati mai tarziu in curs, dar pe scurt, se bazeaza pe a ajunge la un adevar. Noi cand scriem cod in prolog de fapt scriem la fiecare termen **un predicat** (o functie care returneaza bool), si la chestii mai complicate prologul practic cauta o cale catre un adevar prin predicatele pe care le definim noi.

> **Note** (Litere mari vs mici). Se face distinctie intre parametrii cu litere mari si litere mici:
>
> Cele cu litere mici se refera la lucruri concrete (numere, stringuri, etc.).
>
> Cele cu litere mari sunt variabile si ele de fapt pot reprezenta orice termen in spate, fie numere, stringuri, liste etc.
>
> In linia de comanda cand rulam programul daca dam variabile ca parametrii prolog ii va interpreta ca necunoscute si incearca sa gaseasca valori care respecta regulile pentru acele variabile.

Mai jos aveti un exemplu de niste predicate simple (fapte) si un predicat definit prin reguli:

Listing 1.1: Family relations in Prolog

```
% Cazurile de baza (oarecum ca la o functie recursiva)
parent(albert, bob).
parent(albert, betsy).
parent(albert, bill).

parent(alice, bob).
parent(alice, betsy).
parent(alice, bill).

% Reguli extra, prolog va incerca sa ajunga la un adevar pe care l-am definit mai sus
% Separarea prin virgula inseamna 'si' intre toate predicatele enumerate
sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

Este important sa distingem intre operatorii =, ==, 'is':

- Operatorul = de fapt este operatorul de unificare, prolog va incerca sa unifice cei doi termeni cand face backtracking sa caute solutii.
- Operatorul == comparare stricta, nu incearca sa unifice termenii.
- Operatorul 'is' evalueaza expresii matematice.

Exemple:

Listing 1.2: Distinctie operatorii in prolog

```
?- f(X, 2) = f(1, Y).
X = 1,
Y = 2.

?- f(X, 2) == f(1, Y).
false. % Aici se face comparare fara sa incerce sa unifice
    termenii X si Y

?- X = 2 + 3.
X = 2+3.

?- X is 2 + 3.
X = 5.

?- 5 == 2 + 3
false. % Aici termenul '5' nu este egal cu termenul '2 + 3'
```

Liste in Prolog

Listele arata vizual ca si in python: `[1, 2, abc, 4, 5]` dar modul in care lucram cu ele este diferit. In prolog ne folosim de o sintaxa cu Head si Tail:

Listing 1.3: Liste in prolog

```
?- [H|T] = [1, 2, 3, 4, 5].
H = 1,
T = [2, 3, 4, 5].

?- [A, B, C|T] = [1, 2, 3, 4, 5].
A = 1,
B = 2,
C = 3,
T = [4, 5].

?- [_|T] = [1, 2, 3, 4, 5].
T = [2, 3, 4, 5]. % _ este discard character, cand nu iti
    pasa de acel parametru
```

Exercitiu de exemplu (P07 din P99):

Se da o lista sub forma `[1, [2, [3, 4], 5], 6]` si trebuie sa o transformati in `[1, 2, 3, 4, 5, 6]`.

Rezolvare:

Listing 1.4: Exercitiu Flatten List

```
my_flatten([], []). % Cazul de baza

% Cazul cand elementul curent este o lista
my_flatten([H|T], R) :-
    is_list(H),
    my_flatten(H, Fh),
    my_flatten(T, Ft),
    append(Fh, Ft, R).

% Cazul cand elementul curent nu este o lista
my_flatten([H|T], [H|Tr]) :-
    \+ is_list(H),  % Operatorul \+ este operatorul not
    my_flatten(T, Tr).
```

Exercitii propuse

Exercitiul 1

Definiti `my_append/3` echivalent cu append-ul built in.

Rulati si confirmati:

```
?- my_append([1, 2, 3], [4, 5, 6], X)
X = [1, 2, 3, 4, 5, 6].
?- my_append([1, 2, X], [4, 5, 6], [1, 2, 3, 4, 5, 6])
X = 3.
?- my_append([X, 2, Z], [4, 5, 6], [1, 2, 3, 4, Y, 6])
X = 1, Z = 3, Y = 5.
?- my_append(L, [4, 5, 6], [1, 2, 3, 4, 5, 6])
L = [1, 2, 3].
?- my_append(L, [4, 5, 6], [1, 2, 3, 5, 6])
false.
```

Exercitiul 2

Definiti `duplicate/2` care repeta de doua ori elementele dintr-o lista.

Rulati si confirmati:

```
?- duplicate([1, 2, 3], X)
X = [1, 1, 2, 2, 3, 3].
?- duplicate([1, 2, X], [1, 1, 2, 2, 3, 3])
X = 3.
?- duplicate([1, 2, X], [1, 1, 2, 2, 3])
false.
```

Exercitiul 3

Definiti `subset/2` care, dat o lista ca primul paremtru, verifica ca al doilea parametru sa fie o submultime a primei liste.

Rulati si confirmati:

```
?- subset([1, 2, 3], X)
X = []; X = [3]; X = [2]; X = [2, 3]; X = [1]; X = [1, 3]; X = [1,
2]; X = [1, 2, 3]
```

Chapter 2

Continuare Prolog, FNC si FND

2.1 Funcții Boolene

Funcțiile boolene sunt o altă metodă de a descrie semantic formule în mod complet (de a le interpreta, cu toate evaluările posibile), fiind echivalente cu tabelele de adevăr. De fapt un tabel de adevăr definește o funcție booleană.

Fie o formulă φ și mulțimea tutoror variabilelor propoziționale din această formulă $Var(\varphi) = \{v_{i_1}, v_{i_2}, \ldots, v_{i_n}\}$. Conform propoziției 3, evaluările diferite ale unei formule depind doar de valorile pe care le iau cele n variabile din formulă, și pot avea, deci 2^n astfel de evaluări. (fiecare variabilă poate fi 0 sau 1). Ceea ce înseamnă că pentru un tabel de adevăr vor fi necesare 2^n intrări.

Este mult mai ușor să codificăm modul de a evalua o formulă printr-o funcție care ia argument n-upluri de 0 sau 1 (fiecare n-uplu reprezintă valorile pe care le iau, în evaluare, variabilele propoziționale) și care dă rezultat în $\{0, 1\}$.

Definition 3. O funcție booleană este o funcție $F : \{0, 1\}^n \to \{0, 1\}$ unde $n \geq 1$.
spunem că n este numărul variabilelor lui F.

Adică, alegem o evaluare $e : V \to \{0, 1\}$, astfel încât:

$$e(v) = \begin{cases} \varepsilon_k, & v \in Var(\varphi), v = v_{i_k} \\ \\ 0 \text{ sau } 1, & \text{altfel} \end{cases}$$

Funcția booleană asociată lui $F_\varphi : \{0, 1\}^n \to \{0, 1\}$ este definită astfel:

$$F_\varphi(\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n) = e^+(\varphi), \quad \forall\, (\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n) \in \{0, 1\}^n$$

Chiar dacă, matematic acest lucru pare complicat, nu reprezintă decât formalizarea tabelului de adevăr, unde orice argument de tipul $\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n$ este un rând din tabelul de adevăr.

De aici apar mai multe consecințe utile

Proposition 5.

(i) Fie φ o formulă. Atunci:

(a) $\models \varphi$ ddacă F_φ este funcția constantă 1 (Adică, în orice evaluare posibilă a variabilelor din formulă) evaluarea formulei va fi mereu adevărată.).

(b) φ este nesatisfiabilă ddacă F_φ este funcția constantă 0.

(ii) Fie φ, ψ două formule astfel încât $\mathrm{Var}(\varphi) = \mathrm{Var}(\psi)$. Atunci:

(a) $\varphi \models \psi$ ddacă $F_\varphi \leq F_\psi$.

(b) $\varphi \sim \psi$ ddacă $F_\varphi = F_\psi$.

(iii) Există formule diferite φ, ψ a.î. $F_\varphi = F_\psi$.

2.2 Forma normală Conjunctivă și Forma Normală Disjunctivă

2.2.1 Concepte Fundamentale

Un concept fundamental în lucrul cu formule este reprezentat de literali.

Definition 4. Un **literal** este fie o variabilă propozițională (caz în care se numește literal pozitiv), fie negația unei variabile (caz în care se numește literal negativ).
Exemple: $v_0, v_{100}, \neg v_{30}, \ldots$

2.2.2 FNC

O formulă φ este în Forma Normală Conjunctivă dacă este o conjuncție ($\wedge$) de disjuncții ($\vee$) de literali.

Un exemplu este: $(v_0 \vee v_1 \vee v_2) \wedge (v_{10} \vee v_5)$.

Mai general, o formulă în FNC are forma

$$\varphi = \bigwedge_{i=1}^{n} (\bigvee_{j=1}^{k_i} (L_{i,j}))$$

Unde $L_{i,j}$ este un literal.

2.2.3 FND

O formulă φ este în Forma Normală Disjunctivă dacă este o disjuncție ($\wedge$) de conjuncții ($\vee$) de literali.

Un exemplu este: $(v_0 \wedge v_1 \wedge v_2) \vee (v_{10} \wedge v_5)$.

Mai general, o formulă în FNC are forma

$$\varphi = \bigvee_{i=1}^{n} (\bigwedge_{j=1}^{k_i} (L_{i,j}))$$

Unde $L_{i,j}$ este un literal.

Note. Există câteva cazuri speciale

- $v_1 \wedge \neg v_5 \wedge v_{10}$ este atât în FNC (conjuncție de literali), cât și în FND (disjuncția cu un singur element e acel element, deci e o disjuncție de o singură conjuncție).
- $\neg v_3 \vee v_4 \vee v_5$ e atât în FNC cât și în FND (ca mai sus)
- v_0 e atât în FNC cât și în FND.

2.2.4 Aducerea formulelor în FNC și FND

Aici puteți să înțelegeți și la ce ne ajută funcțiile boolene: să facem legătura între formule și formele normale.

Theorem 2. Fie $n \geq 1$ și $H : \{0,1\}^n \to \{0,1\}$ o funcție booleană arbitrară. Atunci o există o formulă φ în FNC și o formulă ψ în FND

$$H = F_\varphi = F_\psi$$

Această teoremă este foarte importantă pentru că ne spune următorul lucru: orice formulă φ am defini (și deci orice funcție booleană asociată ei: H), pot găsi o formulă în FNC și o alta în FND care să fie echivalente semantic cu φ (adică să aibă funcțiile boolene egale).

Formal

Theorem 3. Orice formulă φ este echivalentă cu o formulă φ^{FND} în FND și cu o formulă φ^{FNC} în FNC.

2.2.5 Algoritmul de construcție pentru FNC și FND de la un tabel

Există un algoritm mecanic prin care putem extrage FND și FNC pentru orice funcție booleană definită printr-un tabel de adevăr.

Acest algoritm se bazează pe identificarea liniilor din tabel unde funcția se evaluează la 1 (pentru extragerea FND) și la 0 (pentru extragerea FNC).

Proposition 6. Fie $H : \{0,1\}^n \to \{0,1\}$ o funcție booleană.

(i) Fiecărei evaluări $(\varepsilon_1, \ldots, \varepsilon_n)$ pentru care $H(\varepsilon_1, \ldots, \varepsilon_n) = 1$ îi corespunde o conjuncție de literali, definită ca $C = L_1 \wedge L_2 \wedge \cdots \wedge L_n$, unde $L_i = v_i$ dacă $\varepsilon_i = 1$ și $L_i = \neg v_i$ dacă $\varepsilon_i = 0$. Funcția H este echivalentă cu disjuncția tuturor acestor termeni.

(ii) Fiecărei evaluări $(\varepsilon_1, \ldots, \varepsilon_n)$ pentru care $H(\varepsilon_1, \ldots, \varepsilon_n) = 0$ îi corespunde o disjuncție de literali numită, definită ca $D = L_1 \vee L_2 \vee \cdots \vee L_n$, unde $L_i = v_i$ dacă $\varepsilon_i = 0$ și $L_i = \neg v_i$ dacă $\varepsilon_i = 1$. Funcția H este echivalentă cu conjuncția tuturor acestor termeni.

Aplicație pe un exemplu

Exemplu: Fie $H : \{0,1\}^3 \to \{0,1\}$ descrisă prin tabelul:

ε_1	ε_2	ε_3	$H(\varepsilon_1, \varepsilon_2, \varepsilon_3)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$D_1 = v_1 \vee v_2 \vee v_3$

$D_2 = v_1 \vee v_2 \vee \neg v_3$

$C_1 = \neg v_1 \wedge v_2 \wedge \neg v_3$

$D_3 = v_1 \vee \neg v_2 \vee \neg v_3$

$C_2 = v_1 \wedge \neg v_2 \wedge \neg v_3$

$C_3 = v_1 \wedge \neg v_2 \wedge v_3$

$C_4 = v_1 \wedge v_2 \wedge \neg v_3$

$C_5 = v_1 \wedge v_2 \wedge v_3$

$\varphi = C_1 \vee C_2 \vee C_3 \vee C_4 \vee C_5$ în FND a.î. $H = F_\varphi$.

$\psi = D_1 \wedge D_2 \wedge D_3$ în FNC a.î. $H = F_\psi$.

2.2.6 Algoritmul de aducere a unei formule la FND/FNC

Aducerea unei formule la Forma Normală Conjunctivă (FNC) sau Forma Normală Disjunctivă (FND) se poate realiza și prin transformări sintactice succesive, folosind echivalențe logice fundamentale, fără a fi necesară construirea întregului tabel de adevăr.
Algoritmul mecanic constă în următorii pași:

Pasul 1. Se înlocuiesc implicațiile și echivalențele, folosind regulile:

$$\varphi \to \psi \sim \neg\varphi \vee \psi \quad \text{și} \quad \varphi \leftrightarrow \psi \sim (\neg\varphi \vee \psi) \wedge (\neg\psi \vee \varphi)$$

Pasul 2. Se înlocuiesc dublele negații, folosind $\neg\neg\psi \sim \psi$, și se aplică regulile lui De Morgan pentru a împinge negațiile în interior, direct lângă variabile (transformându-le în literali):

$$\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi \quad \text{și} \quad \neg(\varphi \wedge \psi) \sim \neg\varphi \vee \neg\psi$$

Pasul 3. În funcție de forma dorită, se aplică legile de distributivitate:

- Pentru **FNC**, se aplică distributivitatea disjuncției ($\vee$) față de conjuncție ($\wedge$), pentru a înlocui:

$$\varphi \vee (\psi \wedge \chi) \sim (\varphi \vee \psi) \wedge (\varphi \vee \chi) \quad \text{și} \quad (\psi \wedge \chi) \vee \varphi \sim (\psi \vee \varphi) \wedge (\chi \vee \varphi)$$

- Pentru **FND**, se aplică distributivitatea conjuncției ($\wedge$) față de disjuncție ($\vee$), pentru a înlocui:

$$\varphi \wedge (\psi \vee \chi) \sim (\varphi \wedge \psi) \vee (\varphi \wedge \chi) \quad \text{și} \quad (\psi \vee \chi) \wedge \varphi \sim (\psi \wedge \varphi) \vee (\chi \wedge \varphi)$$

Exemple de aplicare a algoritmului

Exemplul 1 (Aducerea la FNC): Fie formula $\alpha = \neg(p \to q) \vee (r \wedge s)$. Dorim să o aducem la Forma Normală Conjunctivă.

- *Pasul 1 (Eliminarea implicației):*

$$\alpha \sim \neg(\neg p \vee q) \vee (r \wedge s)$$

- *Pasul 2 (Aplicarea legilor De Morgan și eliminarea dublei negații):*

$$\alpha \sim (\neg\neg p \wedge \neg q) \vee (r \wedge s) \sim (p \wedge \neg q) \vee (r \wedge s)$$

- *Pasul 3 (Distributivitatea $\vee$ față de $\wedge$ pentru FNC):* Aplicăm distributivitatea considerând blocul $(p \wedge \neg q)$ ca pe o singură componentă:

$$\alpha \sim ((p \wedge \neg q) \vee r) \wedge ((p \wedge \neg q) \vee s)$$

  Acum aplicăm din nou distributivitatea în interiorul fiecărei paranteze mari:

$$\alpha \sim (p \vee r) \wedge (\neg q \vee r) \wedge (p \vee s) \wedge (\neg q \vee s)$$

Rezultatul final este o conjuncție de disjuncții, formula fiind acum în FNC.

Exemplul 2 (Aducerea la FND): Fie formula $\beta = (p \vee q) \wedge \neg(r \to s)$. Dorim să o aducem la Forma Normală Disjunctivă.

- *Pasul 1 (Eliminarea implicației):*

$$\beta \sim (p \vee q) \wedge \neg(\neg r \vee s)$$

- *Pasul 2 (Aplicarea legilor De Morgan):*

$$\beta \sim (p \vee q) \wedge (\neg\neg r \wedge \neg s) \sim (p \vee q) \wedge (r \wedge \neg s)$$

- *Pasul 3 (Distributivitatea $\wedge$ față de $\vee$ pentru FND):* Aplicăm distributivitatea blocului $(r \wedge \neg s)$ peste disjuncția $(p \vee q)$:

$$\beta \sim (p \wedge (r \wedge \neg s)) \vee (q \wedge (r \wedge \neg s))$$

  Renunțând la parantezele redundante datorită asociativității conjuncției, obținem:

$$\beta \sim (p \wedge r \wedge \neg s) \vee (q \wedge r \wedge \neg s)$$

Rezultatul final este o disjuncție de conjuncții, formula fiind acum în FND.

2.3 Prolog Aritmetica si Exercitii

Aritmetica in Prolog este putin diferita fata de limbajele procedurale si poate fi usor de confundat. Avem urmatorii operatori relevanti: `+, -, *, /, //, is, <, >, >=, =<, =\=, =:=`. Majoritatea sunt intuitivi si fac ceea ce v-ati astepta, dar unele sunt mai confusing si nu fac poate ce credeti ca fac.

Operatorul `is`, utilizat astfel: `X is 2 + 3` sau `X is Y + 1`, va evalua expresia aritmetica din dreapta si va **unifica** rezultatul cu variabila din stanga. Conditie: Toate variabilele din partea dreapta (ex. `Y`) trebuie sa fie **deja instantiate** cu valori numerice.

Operatorul `=:=`, utilizat astfel: `10 =:= 5 + 5`, va evalua expresiile din ambele parti si doar verifica daca rezultatele sunt egale aritmetic. Acesta **nu unifica** variabile; ambele parti trebuie sa poata fi evaluate numeric in momentul apelului.

Similar cu `=:=`, operatorii relationali (`<, >, >=, =<, =\=`) realizeaza comparatii numerice dupa evaluarea ambelor parti si **nu realizeaza unificare**.

Acesti operatori sunt utilizati pentru a restrange universul de solutii, dar nu pot fi folositi pentru a "atribui" valori variabilelor, cu exceptia lui `is`.

Exemple relevante:

```
?- X is 5 + 2.
X = 7.
?- 7 is 5 + 2.
true.
?- 10 is 5 + 2.
false.
?- 10 =:= 5 + 5.
true.
?- X =:= 5 + 2.
ERROR: Arguments are not sufficiently instantiated.
```

Exercitii rezolvate

Note (Sfaturi generale)**.** In rezolvari de obicei e util sa ne gandim la toate cazurile de baza/triviale. Puteti sa scrieti oricate cazuri va vin in minte la inceput ca sa intelegeti problema mai bine, chiar daca sunt acoperite de cazurile recursive (recomandat e sa le stergeti dupa ca posibil sa interfereze in unele cazuri).

In unele cazuri unele probleme ies mult mai usor daca va definiti predicate auxiliare/intermediare (pot avea chiar si acelasi nume atat timp cat au un numar diferit de parametrii). De exemplu:

Listing 2.1: Auxiliary predicates example

```
square(0, _).
square(W, C) :- square(W, W, C).

square(0, _, _).
square(I, W, C) :-
    I > 0,
    I1 is I - 1,
    square(I1, W, C),
    squareRow(W, C).

squareRow(0, _) :- nl.
squareRow(N, C) :-
    N > 0,
    N1 is N - 1,
    squareRow(N1, C),
    write(C).
```

Exercitiul 1

Definiti `suma_lista/2` care face suma elementelor din lista data ca prim argument si pune rezultatul in al doilea argument.

Rulati si confirmati:

```
?- suma_lista([1, 2, 3], 6).
true.
?- suma_lista([1, 2, 3], X).
X = 6.
?- suma_lista([1, 2, X], 6).
ERROR: Arguments are not sufficiently instantiated.
```

Solutie:

```
suma_lista([], 0).
suma_lista([H|T], Sr) :-
    suma_lista(T, S),
    Sr is H + S.
```

Exercitiul 2

Definiti `medie/2` care face media aritmetica a elementelor din lista data ca prim argument si pune rezultatul in al doilea argument.

Rulati si confirmati:

```
?- medie([1, 2, 3, 4], X).
X = 2.5 .
?- medie([1, 2, 3, X], 2.5).
ERROR: Arguments are not sufficiently instantiated.
```

Solutie:

```
% media([], _) :- false.
% media([A], A).
media(L, R) :-
    stats(L, S, N),
    N > 0,
    R is S / N.

stats([], 0, 0).
stats([H|T], S, N) :-
    stats(T, S_coada, N_coada),
    S is H + S_coada,
    N is N_coada + 1.
```

Exercitii propuse

> **Remark.** Incercati sa 'suciti' apelurile in toate directiile cand testati ca sa vedeti cand reuseste si cand nu reuseste sa unifice si sa gaseasca un rezultat (si daca credeti ca ar trebui sa poata, incercati sa solutionati).

Exercitiul 1

Definiti `split/4` care sparge lista primita ca prim argument la pozitia primita ca al doilea argument in doua liste (parametrii 3 si 4).

Rulati si confirmati:

```
?- split([1, 2, 3, 4], 2, [1, 2], [3, 4]).
true.
?- split([1, 2, 3, 4], 2, A, B).
A = [1, 2], B = [3, 4].
?- split(X, 2, [1, 2], [3, 4]).
X = [1, 2, 3, 4].
?- split([1, 2, 3], 0, [], [1, 2, 3]).
true.
?- split([1, 2, 3], X, [1], [2, 3]).
ERROR: Arguments not sufficiently instantiated.
```

Exercitiul 2

Definiti predicatul `filtreaza_peste/3` care extrage numerele dintr-o lista (primul parametru) care sunt peste un anumit prag (al doilea parametru) intr-o alta lista (al treilea parametru).

Rulati si confirmati:

```
?- filtreaza_peste([1, 5, 10, 2, 8], 5, R).
R = [10, 8].
?- filtreaza_peste([10, 20, 30], 40, R).
R = [].
```

Exercitiul 3

Definiti predicatul `intsum/3` care calculeaza suma tuturor numerelor intregi cuprinse intr-un interval [a, b] (primele doua argumente) si pune rezultatul in ultimul argument.

Rulati si confirmati:

```
?- intsum(1, 4, S).
S = 10.
```
$(1 + 2 + 3 + 4 = 10)$

```
?- intsum(5, 5, S).
S = 5.
```

Exercitiul 4

Definiti predicatul `putere/3` care ridica numarul primit ca prim parametru la puterea data ca al doilea parametru si pune rezultatul in ultimul parametru.

Rulati si confirmati:

```
?- putere(3, 3, X).
X = 27.
?- putere(5, 0, X).
X = 1.
?- putere(5, 2, X).
X = 25.
```

2.4 Prolog findall/setof/bagof

In prolog, in mod normal, cand pui un query, sistemul iti returneaza prima solutie si continua cautarea pentru mai multe solutii doar daca vrem noi.

Predicatele `findall`, `bagof`, `setof` sunt folosite atunci cand vrem sa colectam **toate** solutiile intr-o lista.

Diferentele intre ele stau in modul in care trateaza duplicatele, sortarea, lipsa solutiilor si, cel mai important, variabilele libere.

Vom analiza cum se comporta fiecare incepand cu urmatoarele fapte simple:

```
angajat(dana, it).
angajat(alex, it).
angajat(maria, hr).
angajat(alex, it). % Duplicat intentionat
```

findall(sablon, scop, listaRezultat)

Este cel mai simplu si intuitiv si cel mai des folosit in practica. Face exact ce spune: gaseste toate solutiile si le pune intr-o lista.

Pastreaza duplicate, nu sorteaza si **ignora variabilele libere**.

Cand nu gaseste nimic returneaza o lista goala (nu da false).

Exemple:

```
?- findall(Nume, angajat(Nume, pr), R).
R = [].

?- findall(Nume, angajat(Nume, Dept), R).
R = [dana, alex, maria, alex].
```

Se observa cum a ignorat complet variabila libera Dept, a pastrat duplicatul 'alex' si nu a sortat nimic.

bagof(sablon, scop, listaRezultat)

Predicatul `bagof` este putin mai inteligent in sensul ca **grupeaza** rezultatele in functie de variabilele libere (variabilele care sunt in `scop` dar nu sunt in `sablon`).

Pastreaza duplicate, nu sorteaza, si daca exista variabile libere, Prolog **va returna mai multe liste**, cate una pentru fiecare valoare posibila a variabilei libere.

Cand nu gaseste nimic **esueaza** (returneaza false).

Exemple:

```
?- bagof(Nume, angajat(Nume, pr), R).
false.

?- bagof(Nume, angajat(Nume, Dept), R).
Dept = hr,
R = [maria] ;
Dept = it,
R = [dana, alex, alex].
```

De aici putem sa incepem sa si combinam, de exemplu daca vrem o lista de tupluri de tipul:

`[(numeDepartament, [angajati in departament])]`

Putem face:

```
?- findall((Dept, L), bagof(Nume, angajat(Nume, Dept), L), R).
R = [(hr, [maria]), (it, [dana, alex, alex])].
```

setof(sablon, scop, listaRezultat)

Predicatul `setof` este cel mai strict. El modeleaza o multime matematica (un set), si, in general, elementele unei multimi sunt ordonate.

Elimina duplicate, **sorteaza** elementele (alfabetic sau numeric), si similar cu bagof, grupeaza dupa variabilele libere si esueaza cand nu gaseste nimic

Exemple:

```
?- setof(Nume, angajat(Nume, pr), R).
false.

?- setof(Nume, angajat(Nume, Dept), R).
Dept = hr,
R = [maria] ;
Dept = it,
R = [alex, dana].
```

Se observa cum a eliminat duplicatul 'alex' si a sortat listele.

```
?- findall((Dept, L), setof(Nume, angajat(Nume, Dept), L), R).
R = [(hr, [maria]), (it, [alex, dana])].
```

Operatorul ^

Operatorul ^ este folosit in `setof` si `bagof` sa ignore o variabila libera data (similar cu `findall` dar putem controla ce variabile libere sunt ignorate).

Exemple:

```
?- setof(Nume, Dept^angajat(Nume, Dept), R).
R = [alex, dana, maria].
```

In cazul in care avem mai multe variabile libere pe care vrem sa le ignoram putem face:

```
?- setof(Nume, Firma^Dept^lucreaza(Nume, Oras, Firma, Dept), R).
```

Exercitii rezolvate

Exercitiul 1

Avand ca si context urmatoarele fapte:

```
% inventar(Jucator, Obiect, ValoareAur).
inventar(arthas, sabie, 150).
inventar(arthas, potiune_viata, 10).
inventar(arthas, scut, 50).
inventar(jaina, toiag, 200).
inventar(jaina, potiune_mana, 15).
inventar(jaina, potiune_mana, 15). % Are doua potiuni
```

Definiti predicatul `avereJucator/2` care primeste un jucator ca prim parametru si in al doilea parametru se afla aurul total pe care il are acel jucator.

Rulati si confirmati:

```
?- avere_jucator(arthas, R).
R = 210.
?- avere_jucator(jaina, R).
R = 230.
```

Atentie! Ce credeti ca se intampla daca rulam:

```
?- avere_jucator(X, 210).
?- avere_jucator(X, 440).
?- avere_jucator(X, R).
```

Solutii:

Solutia 1 findall:

```
avere_jucator(J, R) :-
    findall(V, inventar(J, _, V), L),
    suma_lista(L, R).
```

In aceasta solutie daca rulam:

```
?- avere_jucator(X, 210).
false.
?- avere_jucator(X, 440).
true.
?- avere_jucator(X, R).
R = 440.
```

De ce? Pentru ca, daca ii dam sa caute jucatorul, atunci J o sa fie o variabila libera. In `findall`, variabilele libere sunt ignorate. Prin urmare, linia cu `findall(V, inventar(J, _, V), L)`, va returna mereu lista cu toate valorile: `[150, 10, 50, 200, 15, 15]` si suma va fi 440.

Solutia 2 bagof:

```
avere_jucator(J, R) :-
    bagof(V, Obj^inventar(J, Obj, V), L),
    suma_lista(L, R).
```

In aceasta solutie daca rulam:

```
?- avere_jucator(X, 210).
X = arthas.
?- avere_jucator(X, 440).
false.
?- avere_jucator(X, R).
X = arthas,
R = 210 ;
X = jaina,
R = 230.
```

De ce? Pentru ca `bagof` va grupa dupa J cand este variabila libera si cand verifica va lua toate gruparile posibile si forma listele: `(arthas, [150, 10, 50])` si `(jaina, [200, 15, 15])` si va lua jucatorul care are suma listei egala cu 210.

Exercitiul 2

Avand ca si context urmatoarele fapte:

```
% mina(X, Y).
mina(1, 1).
mina(1, 3).
mina(2, 2).
mina(3, 1).
```

Definiti predicatul `mine_vecine/3` care primeste coordonatele X si Y (primii doi parametrii) si returneaza numarul de mine vecine prin al treilea parametru.

Rulati si confirmati:

```
?- mine_vecine(1, 2, R).
R = 3.
?- mine_vecine(1, 4, R).
R = 1.
?- mine_vecine(1, 5, R).
R = 0.
```

Solutie:

```
vecin(X1,Y1, X2,Y2) :-
    between(-1, 1, Dx),
    between(-1, 1, Dy),
    \+ (Dx = 0, Dy = 0), % Sau (Dx =\= 0; Dy =\= 0)
    X2 is X1 + Dx,
    Y2 is Y1 + Dy.

mine_vecine(X, Y, R) :-
    findall((Vx,Vy), (vecin(X,Y, Vx,Vy), mina(Vx,Vy)), L),
    length(L, R).
```

Exercitii propuse:

Exercitiul 1

Definiti predicatul `genAllBetween/3` care genereaza toate numerele dintre A si B (primii doi parametrii) si pune rezultatul ca o lista in ultimul parametru. (Hint: folositi `between/3` si `findall/3`)

Rulati si confirmati:

```
?-genAllBetween(4, 10, R).
R = [4, 5, 6, 7, 8, 9, 10]
?-genAllBetween(4, 4, R).
R = [4]
?-genAllBetween(5, 4, R).
R = []
```

Exercitiul 2

Definiti predicatul `aboveMean/2` care gaseste elementele mai mari decat media tuturor elementelor din prima lista si le pune in a doua lista. Implementati folosind una dintre predicatele `findall/3`, `bagof/3` sau `setof/3`

Rulati si confirmati:

```
?-aboveMean([3, 7, 2, 8, 5, 10], R).
R = [7, 8, 10].
?-aboveMean([1, 1, 1, 1, 1, 10], R).
R = [10].
```

Exercitiul 3

Avand ca si context urmatoarele fapte:

```
% nota(Student, Materie, Nota).
nota(andrei, mate1, 8).
nota(andrei, fizica, 4).
nota(andrei, programare1, 10).
nota(maria, mate1, 6).
nota(maria, fizica, 7).
nota(maria, programare1, 10).

% cerinta(MaterieDorita, MaterieCeTrebuiePromovata).
cerinta(mate2, mate1).
cerinta(programare2, programare1).
cerinta(robotica, programare1).
cerinta(robotica, fizica). % Robotica are doua cerinte
```

Definiti predicatul `meterii_eligibile/2` care primeste numele unui student ca primul parametru si in al doilea parametru se afla lista materiilor la care poate sa se inscrie.

Rulati si confirmati:

```
?- materii_eligibile(andrei, X).
X = [mate2, programare2].
?- materii_eligibile(maria, X).
X = [mate2, programare2, robotica].
```